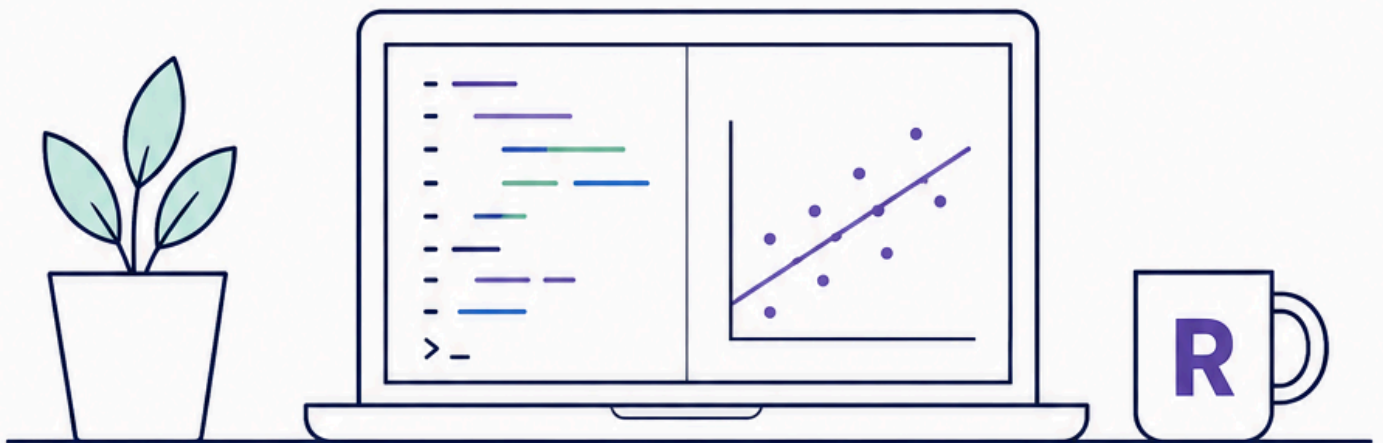
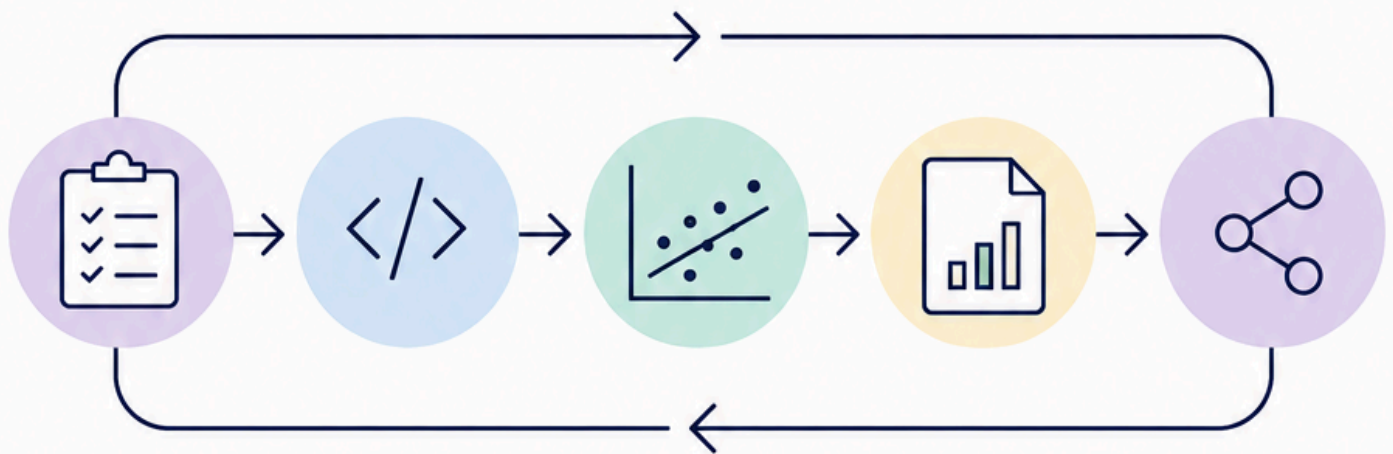


Reproducible Science

A Primer on Workflows
to Support Research

with **R** as One Way to Do It



NELSON ROQUE, PHD

Contents

1	Home	7
2	Preface	9
2.1	What you will learn	9
2.2	Course map	9
2.3	A suggested learning path	9
2.4	About the course	9

I

01 · Foundations

3	What is reproducible science?	13
3.1	Activity: Thought Exercise	13
3.2	What is reproducible science?	13
3.2.1	Benefits of Reproducible Research	13
3.2.2	Reproducible vs Replicable	13
3.3	FAIR Principles	13
4	Reproducible Science Tools & Platforms	15
4.1	Overview	15
4.2	Reproducible Science Tools & Platforms	15
4.3	What is GitHub?	15
4.3.1	GitHub concepts	15
4.4	What is Markdown?	15
4.5	What is R?	16
4.5.1	What is RStudio?	16
4.5.2	What is R Markdown?	16
4.5.3	What are R packages?	16
4.6	What is EndNote?	16
4.6.1	Why EndNote?	16
4.6.2	EndNote downloads	16

II

02 · Working in R

5	R Primer (Syntax and Basic Functions)	19
5.1	Overview of Approach	19
5.2	Installing packages	19
5.2.1	Aside: Some of my favorite packages	19
5.3	Assign values into variables	19
5.4	Check underlying types	19

5.5	File operations	20
5.6	Reading and writing files	20
5.7	Export multiple dataframes into a single Excel workbook	21
5.8	Sanitize column names	21
5.9	Binning data	22
5.10	String manipulations	22
5.11	Generate Random data	22
5.12	Simple stats	22
5.13	Data quality reports	23
5.14	Data integrity	23
6	Anatomy of an R Project	25
6.1	Overview of Approach	25
6.2	Creating an R Project	25
6.3	Project Organization	25
7	Working with Dates	27
7.1	Overview	27
7.2	Load libs	27
7.3	Count wife birthday by day of week	28
7.4	Plot time series of Cinco de Mayo landing on a Tuesday (Taco Tuesday!)	28
7.5	Calculating dates relative to other dates	28

III

03 · Wrangling & analysis

8	Data Wrangling: Special Considerations with Survey Data	31
8.1	Overview	31
8.2	Load data	31
8.3	Split demographics into a separate dataset	31
8.4	Split out survey metadata	31
8.5	Split timer data into a separate dataset	31
8.6	Score TIPI data	32
8.6.1	Create reverse-scoring function	32
8.6.2	Separate TIPI data from main <code>qualtrics_df</code>	32
8.6.3	Transform TIPI from wide to long	32
8.6.4	Recode values	32
8.6.5	Apply Reverse Coding	32
8.6.6	Finally transform to wide format	33
9	Data Wrangling and Visualization of Big Data	35
9.1	Overview	35
9.2	Load libs/packages	35
9.3	Read in data	35
9.4	Inspect dataset features	35
9.5	Create lookup tables	35

9.6	Pre-process data	35
9.7	Joining datasets	36
9.8	Quick descriptives	36
9.9	Filter data	36
9.10	Compute aggregates	36
9.11	Visualize the data	37
9.12	Combining multiple plots	38
9.13	Filter dataset for next plots	39
9.14	Styling your plots (theme_*)	39
9.15	Production plots	40
9.16	Evaluate regions of most change	41
10	Working with Text Data	43
10.1	Background	43
10.2	Overview	43
10.3	Load Libs	43
10.4	Load full dataset	43
10.5	Produce a data quality report	44
10.6	Not all lexicons are equal	44
10.7	Tokenize the text	44
10.8	Compute td-idf statistic	44
10.9	Generate a wordcloud	45
10.10	Conduct simple sentiment analysis	45
10.10.1	Visualize sentiment distribution in each lexicon	45
10.11	Visualize sentiment	46
10.12	Topic modeling analysis	47
10.13	Visualize topic modeling results	47
11	Working with JSON Data	49
11.1	Load data	49
11.2	Pre-process data (tidy it)	49
11.3	Visualize data	50
11.3.1	Keypresses over time	50
11.3.2	Keypresses per question asked by the chatbot	50
12	Working with APIs	51
12.1	Overview	51
12.2	Public APIs as a Learning Tool	51
12.3	The Data	51
12.3.1	Custom function to call API and return data frame	51
13	Regression and Multilevel Modeling	53
13.1	Overview	53
13.2	Load libs	53

13.3	Load data	53
13.4	Pre-process data	53
13.4.1	Create additional columns, remove columns	53
13.4.2	Get unique metadata	54
13.5	Produce simple aggregates	54
13.5.1	Mean center the reaction times	54
13.6	Data visualization	54
13.7	Predictive Modeling	54
13.8	Run simple regression on block means	54
13.8.1	Show resulting model equation	55
13.9	Run as series of incremental multilevel model	55
13.9.1	Compare models	55
13.10	Create publication-ready table of results	55
13.11	Create publication-ready figure of results	55

IV

04 · Keep learning

14	Resources	59
14.1	Recommended R packages	59
14.2	Cheatsheets	59
14.3	Visualization Examples	59
14.4	Datasets	59
14.5	Stats	59
14.6	Books	59
14.7	Blogs	60
14.8	Interactive Learning Tools	60
15	Workshops	61
16	Recordings	63
17	Closing & Acknowledgements	65
17.1	Thank you	65
17.2	Acknowledgments	65
17.3	Questions, Comments, Feedback?	65
	Bibliography	67

1. Home

2. Preface

15 focused chapters
4 learning stages
Free open-access material

2.1 What you will learn

By the end of the course, you will be able to:

- explain why reproducibility matters and apply the FAIR principles;
- organize research as a durable, code-based workflow;
- import, inspect, reshape, join, and visualize real-world data;
- work with dates, survey exports, text, JSON, and public APIs; and
- move from exploratory analysis toward regression and multilevel models.

2.2 Course map

Choose a module or follow the course in order. Each lesson includes focused explanations and code you can adapt to your own work.

Stage 01 · Foundations Why reproducibility? Build the mental model: reproducibility, replication, FAIR principles, and the tools that support open science. Begin with the why →

Stage 02 · Working in R Build a dependable workflow Learn R syntax, file operations, project anatomy, and the date-handling patterns that appear in everyday research. Learn the workflow →

Stage 03 · Wrangling & analysis Turn messy data into evidence Practice with surveys, large datasets, text, JSON, APIs, visualization, regression, and multilevel models. Work with data →

Stage 04 · Keep learning Continue beyond the course Use the curated resources, workshop materials, and recordings to deepen your practice. Browse resources →

2.3 A suggested learning path

Understand the standard. Start with reproducibility and the FAIR principles. Set up the workflow. Learn the tools, R foundations, and project structure. Practice on varied data. Move through survey, time-series, text, JSON, and API examples. Analyze and communicate. Build models, create production-ready outputs, and document your decisions.

How to use this course

You can read the lessons without installing anything. To run the examples, clone the repository, open the R project, and work through the chapters in order. Use the search button or press / whenever you need to find a topic quickly.

2.4 About the course

This open course was created by **Nelson Roque, PhD**, Director of the CASCADE Lab at The Pennsylvania State University. It is designed to give the next generation of scientists a more confident start with reproducible, code-based research.

[Visit the CASCADE Lab](#)



01 · Foundations

3	What is reproducible science?	13
3.1	Activity: Thought Exercise	13
3.2	What is reproducible science?	13
3.3	FAIR Principles	13
4	Reproducible Science Tools & Platforms	15
4.1	Overview	15
4.2	Reproducible Science Tools & Platforms	15
4.3	What is GitHub?	15
4.4	What is Markdown?	15
4.5	What is R?	16
4.6	What is EndNote?	16

3. What is reproducible science?

3.1 Activity: Thought Exercise

- What does reproducible science mean to you?
- Which tools have you previously used?
- Which tools are you most excited to learn?

3.2 What is reproducible science?

Making entire scientific process transparent (when and as allowable by law, grants).

- Sharing experiment and analysis code
- Detailed methods sections
- Sharing stimulus sets
- Being able to walk through a data analysis start (load raw data) to finish (manuscript analyses) in code
- Following FAIR Principles
- Pre-registering hypotheses and analysis plans (e.g., on OSF)

3.2.1 Benefits of Reproducible Research

Sourced from: <https://www.displayr.com/what-is-reproducible-research>

- increased likelihood that the research will be correct
- reproducibility makes it easier to check the research
- it is easier to reproduce the research independently
- easier to extend the research
- reusable code and instruction resulting in increased efficiencies

3.2.2 Reproducible vs Replicable

- Reproducible: when the exact results can be reproduced if given access to the original data, software, or code
- Replicable: research results can be reproduced by independent researchers using different methods.

3.3 FAIR Principles

- **F: Findable:** “The first step in (re)using data is to find them. Metadata and data should be easy to find for both humans and computers. Machine-readable metadata are essential for automatic discovery of datasets and services”
- **A: Accessible:** “Once the user finds the required data, she/he/they need to know how they can be accessed, possibly including authentication and authorisation.”
- **I: Interoperable:** “The data usually need to be integrated with other data. In addition, the data need to interoperate with applications or workflows for analysis, storage, and processing.”
- **R: Reusable:** “The ultimate goal of FAIR is to optimise the reuse of data. To achieve this, metadata and data should be well-described so that they can be replicated and/or combined in different settings.”
- [Learn more about the FAIR Principles](#)
- [FAIR Principles Info PDF](#)

4. Reproducible Science Tools & Platforms

4.1 Overview

Reproducible Science is supported by a large toolkit of software. Below is a highlight of a few important tools that meet most data needs.

4.2 Reproducible Science Tools & Platforms

- [R](#)
 - [LaTeX](#)
 - [Markdown](#)
 - [Python](#), [Anaconda](#)
 - [GitHub](#), [GitHub Pages](#)
 - [Docker](#)
 - VMs (e.g., [VirtualBox](#))
 - Infrastructure as Code (IaC; e.g., [AWS CDK](#))
 - IDEs (e.g., [Visual Studio Code](#))
 - Pre-Registration (e.g., [Open Science Framework](#))
 - Documentation (e.g., [Docusaurus](#))
-

4.3 What is GitHub?

A platform to store and collaborate on code.

[Get GitHub Desktop](#)

[Get GitHub cheatsheets](#)

4.3.1 GitHub concepts

- **Repository**: a logical unit of storage for maintaining code/other assets. Can contain many folders, files.
 - **.gitignore**: a dotfile that specifies which files should not be pushed to GitHub
 - **.gitkeep**: a file (with dot prefix) to specify that a folder should be kept if blank as part of the project structure. This is helpful when creating templates.
 - **commit**: create a record/snapshot of all files (or specific files) at a moment in time. You may tag collaborators, add a title and notes.
 - **push**: 'storing' the result of prior commits
 - **pull**: 'extracting' the result of prior commits
 - **diff**: difference (line-specific) between two commits. Helpful when trying to remember when and where code edits were made.
-

4.4 What is Markdown?

Lightweight markup used to add formatting elements to plaintext documents.

Created by John Gruber in 2004, Markdown is now one of the world's most popular markup languages.

- Using Markdown is different than using a WYSIWYG editor.
- In Word, you click buttons in GUI to apply formatting
- In Markdown, you add syntax to text to apply formatting

For more information, visit: <https://www.markdownguide.org/getting-started/>

4.5 What is R?

Statistical programming language (released ~1993). Similar to S (released ~1976).

Download [R](#)

4.5.1 What is RStudio?

Integrated development environment (IDE) for R.

Download [RStudio](#)

4.5.2 What is R Markdown?

Reproducible notebook (akin to Jupyter Notebooks in Python) where you can write a narrative using Markdown syntax and embed code/plots throughout.

Markdown is a simple formatting syntax for authoring HTML, PDF, MS Word and Powerpoint documents, websites and more. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

This is an R Markdown-based book, using the `bookdown` package.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document.

4.5.3 What are R packages?

Think of CRAN as an equivalent to iOS App Store, or Google Play Store; CRAN is where you download published R packages (directly from R, RStudio).

A developer can also make packages available directly in source code, or via tools like GitHub, for example:

```
install.packages("devtools")
devtools::install_github("nelsonroque/ruf")
```

4.6 What is EndNote?

A citation manager. Especially useful for research projects with large amounts of citations.

Here is a [guide from the UCF library with installation links](#).

4.6.1 Why EndNote?

- no need for manual tracking (deleting, updating) of references
- change citation format at click of a button
- capture PDFs for any citations at click of a button

4.6.2 EndNote downloads

- [Mac](#)
- [Windows](#)

II

02 · Working in R

5	R Primer (Syntax and Basic Functions)	19
5.1	Overview of Approach	19
5.2	Installing packages	19
5.3	Assign values into variables	19
5.4	Check underlying types	19
5.5	File operations	20
5.6	Reading and writing files	20
5.7	Export multiple dataframes into a single Excel workbook	21
5.8	Sanitize column names	21
5.9	Binning data	22
5.10	String manipulations	22
5.11	Generate Random data	22
5.12	Simple stats	22
5.13	Data quality reports	23
5.14	Data integrity	23
6	Anatomy of an R Project	25
6.1	Overview of Approach	25
6.2	Creating an R Project	25
6.3	Project Organization	25
7	Working with Dates	27
7.1	Overview	27
7.2	Load libs	27
7.3	Count wife birthday by day of week	28
7.4	Plot time series of Cinco de Mayo landing on a Tuesday (Taco Tuesday!)	28
7.5	Calculating dates relative to other dates	28

5. R Primer (Syntax and Basic Functions)

5.1 Overview of Approach

This tutorial is written as a guide for the question, “how do I do X in R?”

5.2 Installing packages

```
# install packages from CRAN ----
install.packages(c("tidyverse", "devtools"))
```

5.2.1 Aside: Some of my favorite packages

Install them all in one go with the code below:

```
install.packages(c("tidyverse", "devtools", "qualtRics",
                  "tidytext", "textdata", "topicmodels",
                  "wordcloud", "ggwordcloud", "lme4", "sjPlot",
                  "janitor", "esquisse", "rio",
                  "cowplot", "stringi", "digest"))
```

```
# load libraries ----
library(tidyverse)
set.seed(1013) # for reproducibility
```

5.3 Assign values into variables

```
this_is_double <- 1
this_is_double2 = 1.0
this_is_logical <- TRUE
this_is_date <- as.Date("2020-10-13")
this_is_list <- list(x = rnorm(100), y = rnorm(10))
this_is_dataframe <- data.frame(x = rnorm(100), y = rnorm(100))
this_is_tibble <- tibble::tibble(x = rnorm(100), y = rnorm(100))
this_is_vector <- c(1,2,3)
```

5.4 Check underlying types

```
class(this_is_double)
class(this_is_double2)
class(this_is_logical)
class(this_is_date)
class(this_is_list)
class(this_is_dataframe)
class(this_is_tibble)
class(this_is_vector)
```

```
# generate a sequence ----
seq_1_100 <- seq(1, 100, by=1)
```

```
seq_0_1 <- seq(0, 1, by=0.1)
seq_0_1

# sample from vector of values ----
list_to_sample <- c("A", "1", "2", "B")
sampled_value = sample(list_to_sample, size=1)
sampled_value
sampled_values_replace = sample(list_to_sample, size=10, replace=TRUE)
sampled_values_replace
```

5.5 File operations

```
# list files
all_csv_files = list.files("data",
                           pattern="*.csv",
                           recursive = T,
                           full.names = T)

all_csv_files

all_files = list.files("data",
                      pattern="*",
                      recursive = T,
                      full.names = T)

all_files

# check if directory exists
dir.exists("data")

# create directory
dir.create("test")

# create file if doesn't exist
file <- "test/dataframe.R"
if (file.exists(file)) {
  cat("The file already exists")
} else {
  file.create(file)
  print(paste0("File created: ",file))
}

# remove directory
unlink("test", recursive=TRUE)
```

5.6 Reading and writing files

```
library(readxl)
library(readr)
library(haven)

# write files of
df_for_examples = tibble(x = rnorm(100), y = rnorm(100), study="demo_rnorm_in_r")

write_csv(df_for_examples, "data/demo_fileread.csv")
write_delim(df_for_examples,"data/demo_fileread.txt",delim="\t")
write_sav(df_for_examples, "data/demo_fileread.sav")
write_sas(df_for_examples, "data/demo_fileread.sas")
```

5.7 Export multiple dataframes into a single Excel workbook

```
library(rio)
df_1 = data.frame(x= rnorm(10), dataset=1)
df_2 = data.frame(x= rnorm(10), dataset=2)
df_3 = data.frame(x= rnorm(10), dataset=3)

export(format="xlsx",      file="OneExcelWorkbookWith3Sheet.xlsx",      x=list(df1=df_1,
df2=df_2, df3=df_3))
```

5.8 Sanitize column names

```
library(janitor)

# create demo data
df_janitor_before = tibble::tibble(`A long column Name with Mixed case`=rnorm(20),
                                   `COLUMN_NAME`=rnorm(20),
                                   `GROUP_$`=rnorm(20))
knitr::kable(df_janitor_before)

df_janitor_after = df_janitor_before %>%
  janitor::clean_names()
knitr::kable(df_janitor_after)
```

```
# load data ----

# plaintext data files
library(readr)

# csv
dataset_csv <- read_csv("data/demo_fileread.csv", na = "empty")

# tab-delimited file
dataset_tab <- read_delim("data/demo_fileread.txt", delim = "\t",
                          escape_double = FALSE,
                          trim_ws = TRUE)

# Excel file
library(readxl)
dataset_excel <- read_excel("data/demo_fileread.xlsx", sheet = "Sheet1")

# read SPSS and SAS data file
library(haven)
dataset_sav <- read_sav("data/demo_fileread.sav")
#dataset_sas <- read_sav("data/demo_fileread.sas")
```

```
# dataset operations -----

# stack two or more datasets
df1 = data.frame(x= rnorm(10), dataset=1)
df2 = data.frame(x= rnorm(10), dataset=2)
df3 = data.frame(x= rnorm(10), dataset=3)
df_all = bind_rows(df1, df2, df3)

# join datasets by common identifier
df_link <- data.frame(dataset=c(1,2,3,4),      description=c("mock1",      "mock2",
"mock3", "mock4"))
```

```
df_all_link_innerjoin <- df_all %>% inner_join(df_link)
df_all_link_fulljoin <- df_all %>% full_join(df_link) # end up with 1 extra due to
entry for dataset 4
```

5.9 Binning data

```
n = 50
data_to_bin = data.frame(x = rnorm(n=n,mean=500))
data_binned_label = data_to_bin %>% mutate(x_bin = cut(x,
breaks=c(0,300,400,500,600,Inf)))
data_binned_num = data_to_bin %>% mutate(x_bin = cut(x, breaks=c(0,300,600,800),
labels=FALSE))
data_binned_ord = data_to_bin %>% mutate(x_bin = cut(x, ordered_result=T,
breaks=c(0,300,400,500,600,Inf)))

knitr::kable(head(data_binned_label))
knitr::kable(head(data_binned_num))
```

5.10 String manipulations

```
## Search for data in data
grep("APP", c("APPLES", "APP", "APPLICATION"))
grep("APP", c("APPLES", "APP", "APPLICATION"), value=T)

## Replace data in data
gsub("APPLES", "BANANAS", c("APPLES", "BANANAS"))
gsub("ERR", "ERROR", c("ERR", "ERROR"))
```

```
# stringi stuff
library(stringi)

stri_count_fixed("ACATGAACGGGTACACACTG", "ACA", overlap=TRUE)

x <- c("spam", "")
stri_length(x)

stri_dup(letters[1:5], 1:5)

words <- list(c("spam", "bacon", "sausage", "spam"), c("eggs", "spam"))
stri_join_list(words, sep=", ")
```

5.11 Generate Random data

```
x = rnorm(n=500,mean=300,sd=3)
```

5.12 Simple stats

```
mean(x)
sd(x)
min(x)
max(x)
```

```
quantile(x,c(0.6))  
quantile(x,c(0.95))  
quantile(x,c(0.99))
```

5.13 Data quality reports

```
## Data quality inspection  
knitr::kable(skimr::skim(x)) # one variable  
knitr::kable(skimr::skim(dataset_csv)) # or a whole dataset!  
dqr = skimr::skim(dataset_csv) # save to a variable to later export as csv  
knitr::kable(dqr)
```

5.14 Data integrity

MD5 and SHA256 hashes of data are helpful when making snapshots of data for long-term storage and curation. It ensures that the receiver can verify this hash upon opening the dataset and knowing that no data was lost or corrupted in transmission.

```
library(digest)  
digest("a", algo="md5")  
digest("a", algo="sha256")
```


6. Anatomy of an R Project

6.1 Overview of Approach

R Projects (opened with .RProj files), are a helpful organizing structure for data science projects. By opening the .RProj file, all paths become relative, making it easier to reference files in your code (data/file.csv, vs, C:/Users/nelson/Documents/NestedFolder/Project/data/file.csv).

6.2 Creating an R Project

In RStudio, navigate to: **File > New Project**. Next create a project from a **New Directory** if you do not have a preexisting folder of code, and **Existing Directory** if you do.

6.3 Project Organization

Within the R Project folder, I typically organize the folders and code as follows:

- **data**: folder containing all project data that is processed locally.
- **scripts**: folder containing all scripts to run against your data.
- **output**: folder to store any deliverables created in your **scripts**.
- **reference**: any materials (PDFs, Documents, Presentations) to support your work.

For example, in a research project your folder may look something like this:

```
project_name
project.RProj
data
  demographics.csv
  rt.csv
scripts
  load_install_libs.R
  load_data.R
  preprocess_data.R
  visualize_data.R
  model_data.R
  pipeline.R
output
  data
    tidy_final_dataset.csv
  plots
    figure1.png
    figure1.pdf
    figure1.eps
```

[Click here to download a sample pipeline](#)

7. Working with Dates

7.1 Overview

At some point in time (:p), you'll need to work with date data. This tutorial will illustrate concepts including: converting date formats, extracting date information (e.g., day of week, year), and calculate a date relative to another.

7.2 Load libs

```
library(tidyverse)
```

```
# create date matrix
df <- expand.grid(month=seq(1,12,1),
                  day=seq(1,31,1),
                  year=seq(1930,2030)) %>%
  mutate(mdy = lubridate::mdy(paste0(month,"/",day,"/",year))) %>%
  mutate(dt_month = lubridate::month(mdy),
         dt_day = lubridate::day(mdy),
         dt_year = lubridate::year(mdy),
         dt_week = lubridate::week(mdy),
         dt_yday = lubridate::yday(mdy),
         dt_weekday = lubridate::wday(mdy),
         dt_weekday_l = lubridate::wday(mdy, label=T)) %>%
  filter(!is.na(dt_weekday)) # remove feb 29, 30, 31

knitr::kable(head(df))
```

```
# mark specific dates
df_marked <- df %>%
  mutate(cinco_de_mayo_taco_tuesday = ifelse(dt_month == 5 &
                                              dt_day == 5 &
                                              dt_weekday_l == "Tue", 1, 0),
         friday_13th = ifelse(dt_day == 13 &
                              dt_weekday_l == "Fri", 1, 0),
         erica_bday = ifelse(dt_day == 4 &
                              dt_month == 4, 1, 0))

# segment days Alexa mentioned
all_cinco_de_mayo_taco_tuesday <- df_marked %>%
  filter(cinco_de_mayo_taco_tuesday == 1)

all_friday13th <- df_marked %>%
  filter(friday_13th == 1)

all_erica <- df_marked %>%
  filter(erica_bday == 1) %>%
  filter(year >= 1991)
```

7.3 Count wife birthday by day of week

```
erica_wkday <- all_erica %>%  
  group_by(dt_weekday_l) %>%  
  summarise(n = n())
```

7.4 Plot time series of Cinco de Mayo landing on a Tuesday (Taco Tuesday!)

```
ggplot(df_marked, aes(year, cinco_de_mayo_taco_tuesday)) +  
  geom_tile() +  
  theme_minimal()  
  
ggplot(df_marked, aes(year, cinco_de_mayo_taco_tuesday)) +  
  geom_path() +  
  theme_minimal()  
  
ggplot(erica_wkday, aes(dt_weekday_l, n, group=dt_weekday_l, fill=dt_weekday_l)) +  
  geom_bar(stat="identity")
```

7.5 Calculating dates relative to other dates

! Use real date types

These operations work on types `as.Date` data types (and related `as.POSIXct`), not character data.

```
ref_date = as.Date("2020-10-11")  
paste0("Reference date: ", ref_date)  
paste0("Reference date - 90 days: ", ref_date - 90)  
paste0("Reference date + 90 days: ", ref_date + 90)
```

III

8.1	Overview	31
8.2	Load data	31
8.3	Split demographics into a separate dataset	31
8.4	Split out survey metadata	31
8.5	Split timer data into a separate dataset	31
8.6	Score TIPI data	32
03: Wrangling & analysis		
9.1	Overview	35
9.2	Load libs/packages	35
9.3	Read in data	35
9.4	Inspect dataset features	35
9.5	Create lookup tables	35
9.6	Pre-process data	35
9.7	Joining datasets	36
9.8	Quick descriptives	36
9.9	Filter data	36
9.10	Compute aggregates	36
9.11	Visualize the data	37
9.12	Combining multiple plots	38
9.13	Filter dataset for next plots	39
9.14	Styling your plots (theme_*)	39
9.15	Production plots	40
9.16	Evaluate regions of most change	41
10	Working with Text Data	43
10.1	Background	43
10.2	Overview	43
10.3	Load Libs	43
10.4	Load full dataset	43
10.5	Produce a data quality report	44
10.6	Not all lexicons are equal	44
10.7	Tokenize the text	44
10.8	Compute td-idf statistic	44
10.9	Generate a wordcloud	45
10.10	Conduct simple sentiment analysis	45
10.11	Visualize sentiment	46
10.12	Topic modeling analysis	47
10.13	Visualize topic modeling results	47
11	Working with JSON Data	49
11.1	Load data	49
11.2	Pre-process data (tidy it)	49
11.3	Visualize data	50
12	Working with APIs	51
12.1	Overview	51
12.2	Public APIs as a Learning Tool	51
12.3	The Data	51
13	Regression and Multilevel Modeling	53
13.1	Overview	53
13.2	Load libs	53
13.3	Load data	53
13.4	Pre-process data	53
13.5	Produce simple aggregates	54
13.6	Data visualization	54
13.7	Predictive Modeling	54
13.8	Run simple regression on block means	54
13.9	Run as series of incremental multilevel model	55
13.10	Create publication-ready table of results	55
13.11	Create publication-ready figure of results	55

8. Data Wrangling: Special Considerations with Survey Data

8.1 Overview

In this example, we will work with the `qualtrics` package to read in Qualtrics exports, and apply minimal pre-processing.

8.2 Load data

Read in survey and split StartDate into two columns for easy filtering (we don't need pilot data)

```
library(qualtrics)
library(tidyverse)

# set filename of survey file
fn = "data/demo_qualtrics_export.csv"

# filter out pilot data
PILOT_START_DATE = as.Date("2022-06-21")
qualtrics_df <- read_survey(fn) %>%
  separate(StartDate, into=c("dt_date", "dt_time"), sep=" ", remove=F) %>%
  mutate(is_pilot_data = dt_date < PILOT_START_DATE) %>%
  filter(!is_pilot_data) %>%
  mutate(participant_id = as.character(ResponseId))
```

8.3 Split demographics into a separate dataset

```
df_demo = qualtrics_df %>%
  select(ResponseId, contains("demo"))

knitr::kable(df_demo)# %>% kableExtra::kable_paper(.)
```

8.4 Split out survey metadata

```
df_metadata = qualtrics_df %>%
  select(ResponseId, contains("Date"), contains("duration"), contains("progress"))

knitr::kable(df_metadata)# %>% kableExtra::kable_paper(.)
```

8.5 Split timer data into a separate dataset

```
df_timings = qualtrics_df %>%
  select(ResponseId, contains("timer"))

knitr::kable(df_timings)# %>% kableExtra::kable_paper(.)
```

8.6 Score TIPI data

8.6.1 Create reverse-scoring function

```
reverse_code <- function(df, ques_label) {
  out_df = df %>%
    filter(tipi_question == ques_label) %>%
    mutate(response_n = recode(response,
                                `Disagree strongly` = 7,
                                `Disagree moderately` = 6,
                                `Disagree a little` = 5,
                                `Neither agree nor disagree` = 4,
                                `Agree a little` = 3,
                                `Agree moderately` = 2,
                                `Agree strongly` = 1,
                                .default=-999))

  return(out_df)
}
```

8.6.2 Separate TIPI data from main qualtrics_df

```
# create TIPI dataset -----
tipi_df = qualtrics_df %>%
  select(ResponseId, contains("TIPI"), -contains("_Timer"))
```

8.6.3 Transform TIPI from wide to long

```
tipi_long = tipi_df %>%
  pivot_longer(cols=TIPI_1:TIPI_10, names_to="tipi_question", values_to="response")
```

8.6.4 Recode values

```
tipi_long_r = tipi_long %>%
  mutate(response_n = recode(response,
                              `Disagree strongly` = 1,
                              `Disagree moderately` = 2,
                              `Disagree a little` = 3,
                              `Neither agree nor disagree` = 4,
                              `Agree a little` = 5,
                              `Agree moderately` = 6,
                              `Agree strongly` = 7,
                              .default=-999))
```

8.6.5 Apply Reverse Coding

NOTE: Extraversion: 1, 6R; Agreeableness: 2R, 7; Conscientiousness: 3, 8R; Emotional Stability: 4R, 9; Openness to Experiences: 5, 10R

```
tipi_long_recode2 = reverse_code(tipi_long_r, "TIPI_2")
tipi_long_recode4 = reverse_code(tipi_long_r, "TIPI_4")
tipi_long_recode6 = reverse_code(tipi_long_r, "TIPI_6")
tipi_long_recode8 = reverse_code(tipi_long_r, "TIPI_8")
tipi_long_recode10 = reverse_code(tipi_long_r, "TIPI_10")

tipi_long_nonrecoded = tipi_long_r %>%
```



```

filter(tipi_question %in% c("TIPI_1", "TIPI_3", "TIPI_5", "TIPI_7", "TIPI_9"))

tipi_long_recode_all = bind_rows(tipi_long_nonrecoded,
                                tipi_long_recode2,
                                tipi_long_recode4,
                                tipi_long_recode6,
                                tipi_long_recode8,
                                tipi_long_recode10)

```

8.6.6 Finally transform to wide format

```

tipi_wide_recoded = tipi_long_recode_all %>%
  select(-response) %>%
  pivot_wider(names_from="tipi_question", values_from="response_n") %>%
  rowwise() %>%
  mutate(tipi_extraversion = mean(c(TIPI_1, TIPI_6), na.rm=T)) %>%
  mutate(tipi_agreeableness = mean(c(TIPI_2, TIPI_7), na.rm=T),
         tipi_conscientiousness = mean(c(TIPI_3, TIPI_8), na.rm=T),
         tipi_emostability = mean(c(TIPI_4, TIPI_9), na.rm=T),
         tipi_openexperience = mean(c(TIPI_5, TIPI_10), na.rm=T))

knitr::kable(tipi_wide_recoded %>% select(contains("TIPI_", ignore.case = F)))# %>%
kableExtra::kable_paper(.)

knitr::kable(tipi_wide_recoded %>% select(contains("tipi_", ignore.case = F)))# %>%
kableExtra::kable_paper(.)

```


9. Data Wrangling and Visualization of Big Data

9.1 Overview

For this example, we are working with the Google Community Mobility Reports dataset.

“These Community Mobility Reports aim to provide insights into what has changed in response to policies aimed at combating COVID-19. The reports chart movement trends over time by geography, across different categories of places such as retail and recreation, groceries and pharmacies, parks, transit stations, workplaces, and residential.”

[Learn More](#)

9.2 Load libs/packages

```
library(readr)
library(tidyverse)
```

9.3 Read in data

```
# read in the Google Mobility dataset
google_mobility <- read_csv("https://www.gstatic.com/covid19/mobility/Global_
Mobility_Report.csv")
```

9.4 Inspect dataset features

```
nrow(google_mobility)
ncol(google_mobility)
```

9.5 Create lookup tables

R, as well as some packages, have built in datasets. They can be called with the `data()` function (e.g., `data(state)`). We will leverage a built in dataset with state metadata.

```
data(state)
state_lookup = tibble(sub_region_1 = state.name,
                      us_division = state.division,
                      us_region = state.region,
                      state_land_area = state.area)

# for more state data, look at the following data object (data from 1970s)
state.x77
```

9.6 Pre-process data

- filter data
- add meta-date features (month, year, day, etc)


```
records_per_state = google_mobility_US %>%  
  group_by(sub_region_1) %>%  
  summarise(n_records = n()) %>%  
  arrange(sub_region_1)  
records_per_state  
  
# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>  
  
# group data by weekday  
metrics_month_weekend_US <- google_mobility_US %>%  
  group_by(sub_region_1, dt_year, dt_month_, dt_wkday_l) %>%  
  summarise(avg_change_parks = mean(parks_percent_change_from_baseline, na.rm=T),  
            avg_change_retail_rec =  
mean(retail_and_recreation_percent_change_from_baseline, na.rm=T),  
            avg_change_workplaces = mean(workplaces_percent_change_from_baseline,  
na.rm=T)) %>%  
  inner_join(state_lookup)
```

9.11 Visualize the data

```
# plot summary data across all states
allUS_retail = ggplot(metrics_month_weekend_US %>% filter(us_region %in% c("South",
"West"))), aes(sub_region_1, avg_change_retail_rec)) +
  geom_boxplot() +
  theme_minimal() +
  theme(axis.text.x=element_text(angle=90)) +
  facet_wrap(dt_year~.) +
  labs(x="Month", y="Retail (% Change)")

allUS_parks = ggplot(metrics_month_weekend_US %>% filter(us_region %in% c("South",
"West"))), aes(sub_region_1, avg_change_parks)) +
  geom_boxplot() +
  theme_minimal() +
  facet_wrap(dt_year~.) +
  theme(axis.text.x=element_text(angle=90)) +
  labs(x="Month", y="Parks (% Change)")

allUS_retail
allUS_parks
```

```
jitter_retail <- ggplot(metrics_month_weekend_US, aes(dt_month_,
                                                         avg_change_retail_rec,
                                                         group=us_region,
                                                         color=us_region)) +

#geom_point() +
geom_jitter() +
theme_minimal() +
labs(x="Month", y="Retail & Recreation (% Change)")

smooth_retail = ggplot(metrics_month_weekend_US, aes(dt_month_,
                                                         avg_change_retail_rec,
                                                         group=us_region,
                                                         color=us_region)) +

#geom_point() +
#geom_jitter() +
geom_smooth() +
```

```

theme_minimal() +
labs(x="Month", y="Retail & Recreation (% Change)")

smooth_parks = ggplot(metrics_month_weekend_US, aes(dt_month_,
                                                    avg_change_parks,
                                                    group=us_region,
                                                    color=us_region)) +

#geom_point() +
#geom_jitter() +
geom_smooth() +
theme_minimal() +
labs(x="Month", y="Parks (% Change)")

jitter_retail
smooth_retail
smooth_parks

```

[Click here to learn more about geom_smooth](#)

9.12 Combining multiple plots

```

library(cowplot)

# create a plot of weekly trends
early_covid_retail = ggplot(google_mobility_US_targeted %>%
  filter(dt_month %in% c(1,2,3,4,5)) %>%
  filter(dt_year == 2020), aes(dt_date,
                              retail_and_recreation_percent_change_from_baseline,
                              color=sub_region_1)) +

#geom_point() +
geom_jitter() +
theme_minimal() +
facet_wrap(~sub_region_1) +
labs(x="Month", y="Retail & Rec (% Change)", title="2020")

early_covid_parks = ggplot(google_mobility_US_targeted %>%
  filter(dt_month %in% c(1,2,3,4,5)) %>%
  filter(dt_year == 2020), aes(dt_date,
                              parks_percent_change_from_baseline,
                              color=sub_region_1)) +

#geom_point() +
geom_jitter() +
theme_minimal() +
facet_wrap(~sub_region_1) +
labs(x="Month", y="Parks (% Change)", title="2020")

nxtyear_covid_retail = ggplot(google_mobility_US_targeted %>%
  filter(dt_month %in% c(1,2,3,4,5)) %>%
  filter(dt_year == 2021), aes(dt_date,
                              retail_and_recreation_percent_change_from_baseline,
                              color=sub_region_1)) +

#geom_point() +
geom_jitter() +
theme_minimal() +
facet_wrap(~sub_region_1) +
labs(x="Month", y="Retail & Rec (% Change)", title="2021")

cowplot::plot_grid(early_covid_retail, early_covid_parks, ncol=1)
cowplot::plot_grid(early_covid_retail, nxtyear_covid_retail, ncol=1)

```

```
cowplot::plot_grid(allUS_retail, allUS_parks, ncol=1)
cowplot::plot_grid(smooth_retail, smooth_parks, ncol=1)
```

9.13 Filter dataset for next plots

```
# Variable names -----
#   retail_and_recreation_percent_change_from_baseline = col_double(),
#   grocery_and_pharmacy_percent_change_from_baseline = col_double(),
#   parks_percent_change_from_baseline = col_double(),
#   transit_stations_percent_change_from_baseline = col_double(),
#   workplaces_percent_change_from_baseline = col_double(),
#   residential_percent_change_from_baseline = col_double()

early_covid_period_data = google_mobility_US_w_lu %>%
  filter(dt_month %in% c(1,2,3,4,5,6)) %>%
  filter(dt_year == 2020) %>%
  filter(sub_region_1 %in% c("New York", "California", "Florida", "Wyoming",
    "Texas", "Arizona", "Montana", "South Dakota", "Nevada"))
```

9.14 Styling your plots (theme_*)

```
ggplot(early_covid_period_data, aes(dt_date,
                                     parks_percent_change_from_baseline,
                                     color=sub_region_1)) +

  #geom_point() +
  theme_void() +
  geom_jitter() +
  facet_wrap(~sub_region_1) +
  theme(legend.position = "none")

ggplot(early_covid_period_data, aes(dt_date,
                                     parks_percent_change_from_baseline,
                                     color=sub_region_1)) +

  geom_jitter() +
  facet_wrap(~sub_region_1)

ggplot(early_covid_period_data, aes(dt_date,
                                     parks_percent_change_from_baseline,
                                     color=sub_region_1)) +

  #geom_point() +
  theme_bw() +
  geom_jitter() +
  facet_wrap(~sub_region_1)

ggplot(early_covid_period_data, aes(dt_date,
                                     parks_percent_change_from_baseline,
                                     color=sub_region_1)) +

  #geom_point() +
  theme_dark() +
  geom_jitter() +
  facet_wrap(~sub_region_1)
```

9.15 Production plots

```
ggplot(early_covid_period_data, aes(dt_date,
                                   parks_percent_change_from_baseline,
                                   color=us_region)) +

  #geom_point() +
  theme_bw() +
  geom_jitter() +
  facet_wrap(~sub_region_1) +
  labs(x= "Date", y="Retail & Recreation (% Change)")
```

```
ggplot(early_covid_period_data, aes(dt_date,
                                   parks_percent_change_from_baseline,
                                   color=us_region)) +

  #geom_point() +
  geom_jitter() +
  facet_wrap(~sub_region_1) +
  labs(x= "Date", y="Retail & Recreation (% Change)") +
  theme_bw(base_size = 16) +
  theme(axis.text.x = element_text(angle=90))
```

```
fig1 <- ggplot(early_covid_period_data, aes(dt_date,
                                             retail_and_recreation_percent_change_from_baseline,
                                             color=us_region)) +

  #geom_point() +
  #geom_jitter() +
  geom_smooth() +
  facet_wrap(~sub_region_1) +
  labs(x= "Date", y="Retail & Recreation (% Change)",
       #color="State",
       tag = "Figure 1",
       title = "Google Mobility: Change over Time",
       subtitle = "i.e., Retail & Recreation, early COVID-19 period (01/20 - 06/20)",
       caption = "Google Mobility: Retail & Recreation (% Change) over early COVID
period (01/20 - 06/20)") +
  theme_bw(base_size = 12, base_family = "Arial")

fig2 <- ggplot(early_covid_period_data, aes(dt_date,
                                             parks_percent_change_from_baseline,
                                             color=us_region)) +

  #geom_point() +
  #geom_jitter() +
  geom_smooth() +
  facet_wrap(~sub_region_1) +
  labs(x= "Date", y="Retail & Recreation (% Change)",
       #color="State",
       tag = "Figure 2",
       title = "Google Mobility: Change over Time",
       subtitle = "i.e., Parks, early COVID-19 period (01/20 - 06/20)",
       caption = "Google Mobility: Parks (% Change) over early COVID period (01/20 -
06/20)") +
  theme_bw(base_size = 12, base_family = "Arial")
fig1
fig2
```


9.16 Evaluate regions of most change

```
largest_changes_parks_long <- google_mobility_US %>%
  group_by(sub_region_1, dt_year) %>%
  summarise(avg_change_parks = mean(parks_percent_change_from_baseline, na.rm=T)) %>%
  arrange(sub_region_1)

largest_changes_parks_wide <- largest_changes_parks_long %>%
  #pivot_wider(id_cols=c())
  pivot_wider(names_from = dt_year, values_from = avg_change_parks) %>%
  mutate(change_2021_2020 = `2021` - `2020`)

top10_changes <- largest_changes_parks_wide %>%
  arrange(-change_2021_2020) %>%
  head(n=10)

bottom10_changes <- largest_changes_parks_wide %>%
  arrange(change_2021_2020) %>%
  head(n=10)

largest_changes_parks_long
largest_changes_parks_wide

top10_changes
bottom10_changes
```


10. Working with Text Data

10.1 Background

Text Mining is the process of analyzing text data for key topics, trends, and hidden relationships. Methods include: - Word frequency analysis - Wordclouds - Topic modeling

This collection of methods is especially important since, 80% of the world's data is in an unstructured format.

! The secret is preprocessing

Always consider the data processing implications of your data source. Depending on if you are working with transcriptions from audio or video, or digitized free-entry text, you will want to pre-process your data for misspellings and incorrect transcriptions before proceeding with any text mining.

10.2 Overview

For this example, we will work with product review data from the following open science resource: <https://osf.io/tyue9>

Salminen, J., Kandpal, C., Kamel, A. M., Jung, S., & Jansen, B. J. (2022). Creating and detecting fake reviews of online products. *Journal of Retailing and Consumer Services*, 64, 102771. <https://doi.org/10.1016/j.jretconser.2021.102771>

10.3 Load Libs

```
library(readr)
library(tidyverse)
library(tidytext)
library(textdata)
library(topicmodels)
library(wordcloud)
library(ggwordcloud)
```

10.4 Load full dataset

```
# %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

df <- readr::read_csv("data/fake_reviews_dataset.csv") %>%
  mutate(id = row_number()) %>% # add row id
  select(id, category, everything()) %>%
  mutate(text_clean = stringr::str_replace_all(text_, "[^[:alnum:]]", " ")
  # clean up text a bit

unique(df$category)
unique(df$label)
```

10.5 Produce a data quality report

```
df_dq <- skimr::skim(df)

knitr::kable(df_dq)# %>% kableExtra::kable_minimal(.)
```

10.6 Not all lexicons are equal

Get proportion of stop_words by lexicon (onix, SMART, snowball). Full listing of words [available here](#).

- [Learn more about onix](#)
- [Learn more about SMART](#)
- [Learn more about snowball](#)

```
prop.table(table(stop_words$lexicon))
```

If a word you cared about is in the stop word dictionary you will lose it in your analysis. Use the code below if you need to exclude words from the stopwords dictionary.

```
stop_words_filt = stop_words %>%
  filter(word != "better")
```

```
# Otherwise continue
stop_words_filt = stop_words
```

10.7 Tokenize the text

```
# text mining analyses ----
section_freeresponse_tokens = df %>%
  unnest_tokens(bigram,
    "text_clean",
    token = "ngrams",
    n = 2,
    drop = F) %>%
  separate(bigram, c("word1", "word2"), sep = " ", remove=F) %>%
  filter(!word1 %in% stop_words_filt$word) %>%
  filter(!word2 %in% stop_words_filt$word)
```

10.8 Compute **td-idf** statistic

tf_idf = statistic intended to reflect how important word is to a document

```
# %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

section_fr_1_tf_idf = section_freeresponse_tokens %>%
  count(category, word1) %>%
  bind_tf_idf(word1, category, n) %>%
  arrange(desc(tf_idf))

knitr::kable(head(section_fr_1_tf_idf))# %>% kableExtra::kable_paper(.)
```

```
section_fr_2_tf_idf = section_freeresponse_tokens %>%
  count(label, word1) %>%
```

```

bind_tf_idf(word1, label, n) %>%
  arrange(desc(tf_idf))

knitr::kable(head(section_fr_2_tf_idf))# %>% kableExtra::kable_paper(.)

# vis tf_idf ----
section_fr_1_tf_idf %>%
  group_by(category) %>%
  slice_max(tf_idf, n = 3) %>%
  ungroup() %>%
  ggplot(aes(tf_idf, fct_reorder(word1, tf_idf), fill = category)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~category, ncol = 2, scales = "free") +
  labs(x = "tf-idf", y = NULL)

section_fr_2_tf_idf %>%
  group_by(label) %>%
  slice_max(tf_idf, n = 3) %>%
  ungroup() %>%
  ggplot(aes(tf_idf, fct_reorder(word1, tf_idf), fill = label)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~label, ncol = 2, scales = "free") +
  labs(x = "tf-idf", y = NULL)

```

10.9 Generate a wordcloud

```

# https://towardsdatascience.com/create-a-word-cloud-with-r-bde3e7422e8a
wordcloud::wordcloud(words = section_fr_1_tf_idf %>% pull(word1),
  freq = section_fr_1_tf_idf %>% pull(n),
  min.freq = 300,
  max.words=200,
  random.order=FALSE,
  rot.per=0.1)

```

10.10 Conduct simple sentiment analysis

- nrc. binary “yes”/“no” for categories positive, negative, anger, anticipation, disgust, fear, joy, sadness, surprise, and trust.
- bing. “positive”/“negative” classification.
- AFINN. score between -5 (most negative) and 5 (most positive).
- loughran. “positive”/“negative”/“litigious”/“uncertainty”/“constraining”/“superfluous” classification.

10.10.1 Visualize sentiment distribution in each lexicon

Source code

```

x1 <- get_sentiments(lexicon = "nrc") %>%
  count(sentiment) %>%
  mutate(lexicon = "nrc")
x2 <- get_sentiments(lexicon = "bing") %>%
  count(sentiment) %>%
  mutate(lexicon = "bing")
x3 <- get_sentiments(lexicon = "afinn") %>%
  count(value) %>%
  mutate(lexicon = "afinn") %>%

```

```

mutate(sentiment = as.character(value)) %>%
  select(-value)
# x4 <- get_sentiments(lexicon = "loughran") %>%
#   count(sentiment) %>%
#   mutate(lexicon = "loughran")
x <- bind_rows(x1, x2, x3)

ggplot(x, aes(x = fct_reorder(sentiment, n), y = n, fill = lexicon)) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  labs(title = "Sentiment Counts", x = "", y = "") +
  facet_wrap(~ lexicon, scales = "free")

# %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

# sentiment analysis ----
AFINN <- get_sentiments("afinn")

# more options ---
# get_sentiments(lexicon = c("bing", "afinn", "loughran", "nrc"))

# %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

# merge sentiment with dataset ----
sent_words <- section_freeresponse_tokens %>%
  inner_join(AFINN, by = c(word1 = "word"))

sent_words_fj <- section_freeresponse_tokens %>%
  full_join(AFINN, by = c(word1 = "word"))

# produce various aggregates of sentiment ----
count_words_by_sent = sent_words %>%
  count(category, value, sort = TRUE) %>%
  mutate(n_cut = cut(n, c(0,500,3000,Inf)))

```

10.11 Visualize sentiment

```

ggplot(count_words_by_sent, aes(value, n_cut)) +
  geom_tile() +
  facet_grid(.~category)

avg_sent_by_category = sent_words %>%
  group_by(category) %>%
  summarise(avg_sent = mean(value, na.rm=T),
            sd_sent = sd(value, na.rm=T))

mv = ggplot(avg_sent_by_category, aes(category, avg_sent)) +
  geom_bar(stat="identity") +
  theme(axis.text.x = element_text(angle=90))

sv = ggplot(avg_sent_by_category, aes(category, sd_sent)) +
  geom_bar(stat="identity") +
  theme(axis.text.x = element_text(angle=90))

cowplot::plot_grid(mv, sv, ncol=2)

```

10.12 Topic modeling analysis

Coming soon! Resource: <https://www.tidytextmining.com/topicmodeling.html>

```
ap_data = section_fr_1_tf_idf %>%
  mutate(word1_ = as.character(word1)) %>%
  cast_dtm(category, word1_, n)
```

Topic models in practice use larger k values - k=2 works for this example.

```
ap_lda <- LDA(ap_data, k = 2, control = list(seed = 1234))
ap_topics <- tidy(ap_lda, matrix = "beta")
ap_documents <- tidy(ap_lda, matrix = "gamma")
```

10.13 Visualize topic modeling results

```
ap_top_terms <- ap_topics %>%
  filter(!is.na(term)) %>%
  group_by(topic) %>%
  slice_max(beta, n = 20) %>%
  ungroup() %>%
  arrange(topic, -beta)

ap_top_terms %>%
  mutate(term = reorder_within(term, beta, topic)) %>%
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered()
```

Visualize the terms that had the greatest difference in β between topic 1 and topic 2

```
threshold = .006 # low values return more words
beta_wide <- ap_topics %>%
  mutate(topic = paste0("topic", topic)) %>%
  pivot_wider(names_from = topic, values_from = beta) %>%
  filter(topic1 > threshold | topic2 > threshold) %>%
  mutate(log_ratio_t2_t1 = log2(topic2 / topic1))

ggplot(beta_wide, aes(log_ratio_t2_t1, term)) +
  geom_bar(stat="identity")
```


11. Working with JSON Data

As part of my research program, I am developing a chatbot to assess cognitive function. For this example, we will work with a sample output file from the chatbot, in JSON format.

11.1 Load data

```
library(tidyverse)
cogbot_d = jsonlite::read_json("https://cogtasks.com//x/cogbot/data.json")
```

11.2 Pre-process data (tidy it)

```
# separate data ----
keystroke_df = tibble(cogbot_d$keystrokes)
message_df = tibble(cogbot_d$messages)

# pre-process data ----
message_pp = message_df %>%
  separate(`cogbot_d$messages`,
           sep = ",",
           into = c("ts", "session_uuid", "msg_uuid", "msg_id", "sender", "msg")) %>%
  mutate(ts_c = anytime::anytime(gsub("'", "", gsub("\\[", "", ts)))) %>%
  select(-ts) %>%
  select(ts_c, everything())
head(message_pp)

keystroke_pp = keystroke_df %>%
  separate(`cogbot_d$keystrokes`,
           sep = ",",
           into = c("ts", "session_uuid", "msg_uuid", "event_uuid",
                   "e_ts", "e_loc", "e_key", "e_type", "e_keycode",
                   "e_which", "e_ctrlkey", "e_shiftkey", "e_altkey", "e_metakey",
                   "e_repeat")) %>%
  mutate(e_ts_c = as.numeric(e_ts)) %>%
  mutate(ts_c = anytime::anytime(gsub("'", "", gsub("\\[", "", ts)))) %>%
  select(-ts) %>%
  select(ts_c, everything())
head(keystroke_pp)

# merge dataframes ----

both_pp = keystroke_pp %>%
  full_join(message_pp, by=c("session_uuid", "msg_uuid"))
```

11.3 Visualize data

11.3.1 Keypresses over time

```
ggplot(both_pp, aes(e_ts_c, e_key)) +  
  geom_point() +  
  theme_bw() +  
  theme(axis.text.x = element_text(angle=90))
```

11.3.2 Keypresses per question asked by the chatbot

```
ggplot(both_pp, aes(e_ts_c, e_key)) +  
  geom_point() +  
  theme_bw() +  
  theme(axis.text.x = element_text(angle=90)) +  
  facet_grid(.~msg)
```

12. Working with APIs

12.1 Overview

APIs make the world go round. From airlines, to your groceries, APIs are charged with securely and efficiently shipping data back and forth between businesses and their consumers.

This example will demonstrate how to interact with a set of APIs that cover a range of possible use cases (e.g., paginated and non-paginated returns, authentication).

⚠ Be mindful of limits

When connecting to APIs for data, be mindful of the frequency of requests you make. With overuse, depending on the API, you may be throttled, blocked, or limited for a period of time. Check API documentation for posted limits (e.g., 1000 requests per month).

12.2 Public APIs as a Learning Tool

Below is a list of public APIs you may consider for your learning:

- <https://github.com/public-apis/public-apis>
- [Randomly Generated Data](#)

12.3 The Data

For this learning exercise, we will work with an API that generates random data. We will get 10 records of faux user records.

https://random-data-api.com/api/internet_stuff/random_internet_stuff

12.3.1 Custom function to call API and return data frame

```
call_api <- function(api) {  
  return(as.data.frame(jsonlite::read_json(api)))  
}
```

```
library(tidyverse)  
  
n_records = 10  
results = list()  
for(i in 1:n_records) {  
  results[[i]] = call_api("https://random-data-api.com/api/internet_stuff/random_internet_stuff")  
}  
results_df = bind_rows(results)
```

```
library(knitr)  
library(kableExtra)  
knitr::kable(results_df) # %>% kableExtra::kable_minimal(.)
```


13. Regression and Multilevel Modeling

13.1 Overview

This example uses data from a reaction time experiment where observers searched for a green circle amongst rectangles, and were asked to report if a line inside the green circle was vertical or horizontal. On some trials, a red rectangle was present, i.e, a distractor. The experiment was conducted in two ways: (1) a random ordering of distractor present and absent trials, and (2) a blocked ordering of distractor present and absent trials. With blocked administration, presumably participants had time to habituate to the distractor and would have a smaller response time decrement as compared with the random presentation condition.

In this example analysis, we will first take a simple approach, and conduct a linear regression, on participant block-level means of response time. Thereafter we will apply multilevel modeling to better capture the nesting and variability inherent in the trial-level data.

13.2 Load libs

```
library(readr)
library(tidyverse)
library(lme4)
```

13.3 Load data

```
exp_df <- read_csv("data/exp1.csv")

# show data structure
head(exp_df)
```

13.4 Pre-process data

```
# filter
exp_pp <- exp_df %>%
  filter(!is.na(PARTICIPANT))
```

13.4.1 Create additional columns, remove columns

```
exp_pp_ = exp_pp %>%
  mutate(is_practice = ifelse(practice == "yes", 1, 0)) %>%
  mutate(TRIAL = count_trial_sequence) %>%
  select(-count_trial_sequence, -response_time, -practice) %>%
  select(datetime, PARTICIPANT, BLOCK, TRIAL, is_practice, correct, RT, everything())
  %>%
  mutate(EXP_BLOCK_RANDOM = ifelse(isRandom == "blocked", 0, 1))
```

13.4.2 Get unique metadata

```
metadata = exp_pp_ %>%
  select(datetime, contains("DT_"), PARTICIPANT, contains("EXP_")) %>%
  distinct()
```

13.5 Produce simple aggregates

```
# overall means and record count by participant
part_means = exp_pp_ %>%
  group_by(PARTICIPANT, datetime) %>%
  summarise(mean_rt = mean(RT, na.rm=T),
            sd_rt = sd(RT, na.rm=T),
            q10_rt = quantile(RT, probs=0.1, na.rm=T),
            q90_rt = quantile(RT, probs=0.9, na.rm=T),
            n_correct = sum(correct),
            n = n()) %>%
  inner_join(metadata)

# overall means by block
part_means_byblock = exp_pp_ %>%
  group_by(PARTICIPANT, datetime, BLOCK, is_practice, distractor) %>%
  summarise(mean_rt = mean(RT, na.rm=T),
            sd_rt = sd(RT, na.rm=T),
            q10_rt = quantile(RT, probs=0.1, na.rm=T),
            q90_rt = quantile(RT, probs=0.9, na.rm=T),
            n_correct = sum(correct),
            n = n()) %>%
  inner_join(metadata)
```

13.5.1 Mean center the reaction times

```
exp_m = exp_pp_ %>%
  inner_join(part_means) %>%
  mutate(imean_c_RT = RT - mean_rt)
```

13.6 Data visualization

```
ggplot(part_means_byblock %>% filter(!is_practice), aes(BLOCK, mean_rt, group=BLOCK,
color=as.factor(EXP_BLOCK_RANDOM))) +
  geom_boxplot() +
  facet_grid(. ~ EXP_BLOCK_RANDOM) +
  theme_bw() +
  labs(x= "Experiment Block", y = "Mean Response Time (ms)")
```

13.7 Predictive Modeling

13.8 Run simple regression on block means

```
fit = lm(mean_rt ~ n_correct + distractor, data=part_means_byblock)
summary(fit)
```

13.8.1 Show resulting model equation

```
# show model equation
library(equationomatic)
extract_eq(fit, use_coefs=TRUE, wrap=TRUE)
```

13.9 Run as series of incremental multilevel model

```
# run models, starting with unconditional means model
fitm_0 = lme4::lmer(RT ~ 1 + (1|PARTICIPANT),
                    data=exp_pp_)
fitm_1 = lme4::lmer(RT ~ TRIAL + (1|PARTICIPANT),
                    data=exp_pp_)
fitm_2 = lme4::lmer(RT ~ TRIAL + is_practice + (1|PARTICIPANT),
                    data=exp_pp_)
fitm_3 = lme4::lmer(RT ~ TRIAL + is_practice + correct + (1|PARTICIPANT),
                    data=exp_pp_)
fitm_4 = lme4::lmer(RT ~ TRIAL + is_practice + correct +
                    distractor +
                    (1|PARTICIPANT),
                    data=exp_pp_)
fitm_5 = lme4::lmer(RT ~ TRIAL + is_practice + correct +
                    distractor +
                    display_size +
                    EXP_BLOCK_RANDOM +
                    (1|PARTICIPANT),
                    data=exp_pp_)
```

```
extract_eq(fitm_5, use_coefs=TRUE, wrap=TRUE)
```

13.9.1 Compare models

```
# compare models
anova(fitm_0, fitm_1, fitm_2, fitm_3, fitm_4, fitm_5)
```

13.10 Create publication-ready table of results

```
sjPlot::tab_model(fitm_0, fitm_5)
```

13.11 Create publication-ready figure of results

```
sjPlot::plot_model(fitm_5, type="pred")
```


IV 04 · Keep learning

14	Resources	59
14.1	Recommended R packages	59
14.2	Cheatsheets	59
14.3	Visualization Examples	59
14.4	Datasets	59
14.5	Stats	59
14.6	Books	59
14.7	Blogs	60
14.8	Interactive Learning Tools	60
15	Workshops	61
16	Recordings	63
17	Closing & Acknowledgements	65
17.1	Thank you	65
17.2	Acknowledgments	65
17.3	Questions, Comments, Feedback?	65
17.3	Bibliography	67

14. Resources

14.1 Recommended R packages

Install them all in one go with the code below:

```
install.packages(c("tidyverse", "devtools", "qualtRics",  
                  "tidytext", "textdata", "topicmodels",  
                  "wordcloud", "ggwordcloud", "lme4", "sjPlot",  
                  "janitor", "esquisse", "rio",  
                  "cowplot", "stringi", "digest"))
```

14.2 Cheatsheets

- [List of Cheatsheets](#)
- [ggplot2](#)
- [dplyr](#)
- [tidyr](#)
- [Data import with readr, readxl, and googlesheets4 cheatsheet](#)
- [Apply functions with purrr](#)
- [String manipulation](#)
- [Working with dates and times](#)
- [RMarkdown](#)

14.3 Visualization Examples

- [R-Charts](#)
- [Design your own Grid Plot](#)

14.4 Datasets

- [Dr. Roque's list of Datasets](#)
- [fivethirtyeight](#)
- [UK Baby Names](#)
- [More baby names](#)
- [TidyTuesday](#)
- [LOTR](#)
- [Most raw data from gapminder](#)
- [Untidy data](#)
- [Data Science in a Box](#)
- [Data visualization](#)

14.5 Stats

- [Intro to R Stats](#)
- [Multilevel Modeling Primer 1](#)
- [Multilevel modeling Primer 2](#)

14.6 Books

- [R for Data Science](#)
- [R Graphics Cookbook](#)

- [Efficient R Programming](#)
- [Advanced R](#)
- [Text Mining with R](#)
- [Reproducible Analyses with R](#)

14.7 Blogs

- [Revolution Analytics](#)
- [RBloggers](#)

14.8 Interactive Learning Tools

- [Swirl - Interactive R](#)
- [DataCamp](#)
- [CodeAcademy](#)

15. Workshops

- Intro to Reproducible Science: July 6, 8, 11th, 2022; 11am to 2pm
 - **Submit Questions:** [Submit your questions here](#)
 - **Location:** In person at UCF; in PSY301Q
 - **Format:** Live, with recordings available for later viewing.
 - **Objectives**
 - Describe various tools and techniques supportive of open and reproducible science.
 - List and describe the FAIR Principles (<https://www.go-fair.org/fair-principles>)
 - Develop a code-only pipeline to allow reproducibility of data prep and analyses.
 - Develop a long-term learning plan for practicing reproducible science tools and techniques.
 - **Day 1**
 - What is Reproducible Science?
 - Reproducible & FAIR Data Workflows
 - Tools Supporting Reproducible Science
 - Overview of available tools
 - Skill 1: Using Endnote for Reference Management
 - Skill 2: Using Git (and Github) for code management and collaboration
 - Orientation to R, RStudio, RMarkdown
 - Skill 3: R syntax primer
 - Data Science: Latest trends
 - Long-term Learning Recommendations
 - **Day 2**
 - Data wrangling and visualization of Big Data
 - Skill 1: Data wrangling the Google Mobility dataset
 - Reproducible survey research
 - Qualtrics survey design tips
 - Skill 2: Data wrangling Qualtrics data
 - Working with JSON data
 - Skill 3: cleaning and visualizing keystroke JSON data
 - **Day 3**
 - Text mining
 - Skill 1: word and bigram frequency analysis
 - Skill 2: generating wordclouds
 - Skill 3: sentiment analysis
 - Interacting with APIs and JSON data
 - Skill 4: querying API for results and data aggregation
 - Closing Discussion & Q/A

16. Recordings

- Intro to Reproducible Science: July 6, 8, 11th, 2022; 11am to 2pm
 - **Day 1:** Recording coming soon!
 - **Day 2:** Recording coming soon!
 - **Day 3:** Recording coming soon!

17. Closing & Acknowledgements

17.1 Thank you

Thank you for your time and engagement in learning via this webcourse. I hope you've learned a few useful skills that you can continue to develop over time.

17.2 Acknowledgments

I want to thank a number of people who made this possible:

- My wife, daughter
- Dr. Mark Neider
- Dr. Florian Jentsch
- [Dr. Cary Stothart](#)
- Dr. Justin Mason
- Context Lab and CASCADE Lab students

17.3 Questions, Comments, Feedback?

Feel free to email me at nur375@psu.edu if you have any questions, comments, or feedback.

Bibliography